JobSheet - Week 2 - Object, Class & Encapsulation

Nama : **Raza Mulki Firdaus**

NIM : **251524120**

Kelas : **1D**

Repo GitHub : **https://github.com/zaaraza/Teknik-Pemrograman-**

Instruksi Pengerjaan:

1. Kerjakan 2 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W2_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W2_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Evaluasi & Refactoring Class Restaurant

Soal Praktikum
Diketahui terdapat dua file program yang digunakan untuk menyimpan daftar menu makanan dan stoknya, yaitu `Restaurant.java` & `RestaurantMain.java`.

1. Restaurant.java

```
public class Restaurant {
    public String[] nama_makanan;
    public double[] harga_makanan;
    public int[] stok;
    public static byte id = 0;

    public Restaurant() {
        nama_makanan = new String[10];
        harga_makanan = new double[10];
        stok = new int[10];
    }
}

public void tambahMenuMakanan(String nama, double harga, int stok) {
```

```
    this.nama_makanan[id] = nama;
    this.harga_makanan[id] = harga;
    this.stok[id] = stok;
}

public void tampilMenuMakanan() {
    for (int i = 0; i <= id; i++) {
        if (!isOutOfStock(i)) {
            System.out.println(
                nama_makanan[i] + "[" + stok[i] + "]" + "\tRp. " + harga_makanan[i]
            );
        }
    }
}

public boolean isOutOfStock(int id) {
    if (stok[id] == 0) {
        return true;
    } else {
        return false;
    }
}

public static void nextId() {
    id++;
}
```

2. RestaurantMain.java

```
public class RestaurantMain {
    public static void main(String[] args) {
        Restaurant menu = new Restaurant();

        menu.tambahMenuMakanan("Pizza", 250000, 20);
        Restaurant.nextId();

        menu.tambahMenuMakanan("Spaghetti", 80000, 20);
        Restaurant.nextId();

        menu.tambahMenuMakanan("Tenderloin Steak", 60000, 30);
        Restaurant.nextId();

        menu.tambahMenuMakanan("Chicken Steak", 45000, 30);

        menu.tampilMenuMakanan();
    }
}
```

A. Instruksi - Analisis Desain Class:

Amati kode program yang diberikan, kemudian jawab pertanyaan berikut:

1. Apakah class Restaurant sudah menerapkan Encapsulation dengan benar? Anda dapat mengacu pada Buku Ajar Tekpro (Teori) - Chapter 2.4 Enkapsulasi (Lihat di Buku Acuan pada Google Classroom) atau Buku Java Core Design Hint Chapter 4.10.
2. Apakah attribute yang diterapkan saat ini aman dari akses secara

langsung?

3. Apakah terdapat pelanggaran prinsip OOP (misalnya public attribute)?

B. Instruksi - Perbaikan Kode Program:

Lakukan perbaikan desain class dengan ketentuan:

1. Semua attribute harus bersifat private
2. Akses data dilakukan melalui getter dan setter
3. Validasi stok (stok tidak boleh negatif)
4. Pengembangan Fitur (Mini Case) Tambahkan fitur berikut:
   a) Pemesanan menu
   b) Stok otomatis berkurang setelah pemesanan
   c) Pesan ditolak jika stok tidak mencukupi

Source Code

```java
public class Restaurant {
    // Mengubah public menjadi private untuk enkapsulasi data
    private String[] namaMakanan;
    private double[] hargaMakanan;
    private int[] stok;
    private int id; // Mengubah dari yang tadinya static menjadi instance variable untuk menghindari
                    // masalah pada penambahan menu

    public Restaurant() {
        namaMakanan = new String[10];
        hargaMakanan = new double[10];
        stok = new int[10];
        id = 0;
    }

    // getters dan setters untuk mengakses dan memodifikasi data dengan validasi
    // sederhana
    public int getId() {
        return id;
    }

    public String getNamaMakanan(int index) {
        return namaMakanan[index];
    }

    public void setNamaMakanan(int index, String nama) {
        if (nama != null && !nama.isEmpty()) {
            namaMakanan[index] = nama;
        }
    }

    public double getHargaMakanan(int index) {
        return hargaMakanan[index];
    }

    public void setHargaMakanan(int index, double harga) {
        if (harga > 0) {
            hargaMakanan[index] = harga;
        } else {
            System.out.println("Harga harus lebih dari 0");
        }
    }

    public int getStok(int index) {
        return stok[index];
    }

    public void setStok(int index, int stokBaru) {
        if (stokBaru >= 0) {
            stok[index] = stokBaru;
        } else {
            System.out.println("Stok tidak boleh negatif");
        }
    }

    public void tambahMenuMakanan(String nama, double harga, int stokAwal) {
        setNamaMakanan(id, nama);
        setHargaMakanan(id, harga);
        setStok(id, stokAwal);
        id++;
    }

    public void tampilMenuMakanan() {
        for (int i = 0; i < id; i++) {
            if (getStok(i) > 0) {
                System.out.println(
                        getNamaMakanan(i) + " [" + getStok(i) + "] Rp. " + getHargaMakanan(i));
            }
        }
    }

    public void pesanMakanan(String nama, int jumlah) {
        for (int i = 0; i < id; i++) {
            if (getNamaMakanan(i).equalsIgnoreCase(nama)) {

                if (getStok(i) >= jumlah) {
                    setStok(i, getStok(i) - jumlah);
                    System.out.println("Pesanan berhasil. Sisa stok: " + getStok(i));
                } else {
                    System.out.println("Pesanan ditolak. Stok tidak mencukupi");
                }
                return;
            }
        }
        System.out.println("Menu tidak ditemukan");
    }
}
```

```java
public class RestaurantMain {
    public static void main(String[] args) {

        Restaurant menu = new Restaurant();

        menu.tambahMenuMakanan("Pizza", 250000, 20);
        menu.tambahMenuMakanan("Spaghetti", 80000, 20);
        menu.tambahMenuMakanan("Tenderloin Steak", 60000, 30);
        menu.tambahMenuMakanan("Chicken Steak", 45000, 30);

        System.out.println("=== Daftar Menu Awal ===");
        menu.tampilMenuMakanan();

        System.out.println("\n=== Proses Pemesanan ===");
        menu.pesanMakanan("Pizza", 5);
        menu.pesanMakanan("Spaghetti", 25);
        menu.pesanMakanan("Chicken Steak", 10);

        System.out.println("\n=== Menu Setelah Pemesanan ===");
        menu.tampilMenuMakanan();
    }
}
```

Screenshot Hasil

```
=== Daftar Menu Awal ===
Pizza [20] Rp. 250000.0
Spaghetti [20] Rp. 80000.0
Tenderloin Steak [30] Rp. 60000.0
Chicken Steak [30] Rp. 45000.0

=== Proses Pemesanan ===
Pesanan berhasil. Sisa stok: 15
Pesanan ditolak. Stok tidak mencukupi
Pesanan berhasil. Sisa stok: 20

=== Menu Setelah Pemesanan ===
Pizza [15] Rp. 250000.0
Spaghetti [20] Rp. 80000.0
Tenderloin Steak [30] Rp. 60000.0
Chicken Steak [20] Rp. 45000.0
```

Penjelasan Permasalahan dan Solusi

A. Analisis Desain Class

1. Menurut saya, class Restaurant belum menerapkan Encapsulation dengan benar. Ini disebabkan oleh banyaknya objek yang sifatnya public sehingga belum ter-enkapsulasi. Akibatnya adalah semua objek yang sifatnya public bisa diakses dan diubah langsung dari luar class.

2. Semua attribute seperti nama_makanan, harga_makanan, stok, dan id masih menggunakan access modifier public. Artinya, class lain dapat mengakses dan mengubah isi data secara langsung tanpa melalui kontrol dari dalam class. Kondisi ini berisiko menimbulkan inkonsistensi data. Misalnya, class lain dapat mengubah stok menjadi negatif atau mengubah harga tanpa validasi. Jadi, menurut saya ini belum aman.

3. Ada, terdapat pelanggaran prinsip OOP karena penggunaan public attribute dan static yang kurang tepat. Variabel static berarti milik class, bukan milik object. Jika suatu saat dibuat lebih dari satu object Restaurant, maka semua object akan berbagi nilai id yang sama. Hal ini bisa menimbulkan bug.

B. Perbaikan Kode Program (Source code dan Screenshot hasil bisa dilihat diatas)

1. Semua Attribute sudah diubah ke Private
2. Akses data sudah dilakukan melalui getter dan setter

3. Validasi stok sudah ditambahkan 4. Pengembangan fitur berupa pemesanan menu, stok berkurang saat sudah dilakukan pemesanan, dan pesan ditolak jika stok tidak mencukupi
Nama Teman Hal yang Dibantu (Opsional)
-

No. 2 Interaksi Object dan Struktur Class dalam Java

Soal Praktikum
Pada pertemuan sebelumnya, mahasiswa telah mempelajari konsep dasar dalam Pemrograman Berorientasi Objek (Object-Oriented Programming/OOP) yang menjadi fondasi utama dalam pengembangan aplikasi berbasis Java. 1. *Object* Object merupakan representasi nyata dari suatu entitas yang memiliki data (attribute) dan perilaku (method). Object dibuat dari sebuah class dan digunakan untuk menjalankan proses dalam program. 2. *Class* Class adalah cetak biru (blueprint) untuk membuat object. Class mendefinisikan struktur data dan perilaku yang dimiliki oleh object, sehingga memungkinkan pembuatan object dengan karakteristik yang sama. 3. *Message* Message menggambarkan proses komunikasi antar object. Dalam Java, message diwujudkan dalam bentuk pemanggilan method pada suatu object untuk meminta object tersebut melakukan suatu aksi. 4. *Encapsulation* Encapsulation adalah prinsip OOP yang digunakan untuk melindungi data dengan membatasi akses langsung terhadap attribute. Penerapan

encapsulation dilakukan menggunakan access modifier dan accessor method (getter dan setter).

Pada pertemuan kali ini, mahasiswa akan mempelajari praktik beberapa konsep lanjutan dalam Pemrograman Berorientasi Objek menggunakan Java yang berfokus pada interaksi antar object serta struktur dan organisasi program.

1. *Relationship Between Class*
   Membahas hubungan antar class dalam sebuah sistem, seperti bagaimana satu class dapat menggunakan atau memiliki object dari class lain untuk membentuk sistem yang lebih kompleks.
2. *Static Field & Method*
   Mempelajari penggunaan field dan method yang bersifat static, yaitu milik class dan bukan milik object, serta memahami kapan dan mengapa konsep static digunakan.
3. *Method Parameters*
   Membahas cara pengiriman data ke dalam method melalui parameter, termasuk penggunaan object sebagai parameter untuk memungkinkan interaksi antar object.
4. *Object Construction*
   Mempelajari proses pembuatan object melalui constructor, termasuk inisialisasi attribute dan pengaturan kondisi awal object saat pertama kali dibuat.
5. *Packages*
   Membahas pengelompokan class ke dalam package untuk mengatur struktur program, meningkatkan keterbacaan kode, dan menghindari konflik nama class.
6. JAR
   Mempelajari konsep Java Archive (JAR) sebagai media untuk mengemas dan mendistribusikan aplikasi Java agar dapat dijalankan atau dibagikan dengan lebih mudah.

Ketentuan Praktikum:
Pada praktikum ini, mahasiswa diminta untuk memahami contoh program employee dan mengembangkan aplikasi tersebut dengan menerapkan beberapa konsep lanjutan dalam Pemrograman Berorientasi Objek menggunakan Java.

A. Instruksi - Tulis Ulang Kode Program Employee

1. Kelas Department

```
package id.ac.polban.employee.model;

public class Department {
 private String name;

 public Department(String name) {
    this.name = name;
 }

 public String getName() {
    return name;
 }
```

```
 public void setName(String name) {
    this.name = name;
 }
}
```

2. Kelas Employee

```
package id.ac.polban.employee.model;

public class Employee {
    private int id;
    private String name;
    private Department department;
    private EmploymentType type;
    private double salary;

    public Employee(int id, String name, Department department,
EmploymentType type, double salary) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.type = type;
        this.salary = salary;
    }

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Department getDepartment() {
        return department;
    }
    public void setDepartment(Department department) {
        this.department = department;
    }
    public EmploymentType getType() {
        return type;
    }
    public void setType(EmploymentType type) {
        this.type = type;
    }
    public double getSalary() {
        return salary;
    }
    public void setSalary(double salary) {
        this.salary = salary;
    }
}
```

3. Kelas EmployeeType

```
package id.ac.polban.employee.model;

public class EmploymentType {
```

```
    private String type;

    public EmploymentType(String type) {
        this.type = type;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }
}
```

4. Kelas EmployeeService

```
package id.ac.polban.employee.service;

import java.util.HashMap;
import java.util.Map;

import id.ac.polban.employee.model.*;

// mengelola operasi yang berkaitan dengan data dan aturan bisnis
public class EmployeeService {
    private Map<Integer, Employee> employees = new HashMap<>();

    public void addEmployee(Employee emp) {
        employees.put(emp.getId(), emp);
    }

    public Employee getEmployee(int id) {
        return employees.get(id);
    }

    public void raiseSalary(int id, double percent) {
        Employee emp = employees.get(id);
        if (emp != null) {
            emp.setSalary(emp.getSalary() * (1 + percent/100));
        }
    }
}
```

B. Instruksi - Lengkapi Studi Kasus, Rancangan Class Diagram & Penjelasan:

Adapun ketentuan dan tugas praktikum adalah sebagai berikut:

1. Lengkapi studi kasus yang telah dibuat dengan menerapkan penggunaan *static field* dan *static method* secara tepat!
2. Terapkan konsep package dengan membuat minimal dua package, yaitu:
    - `id.ac.polban.employee.model`
    - `id.ac.polban.employee.service`
2. Buat diagram kelas (**class diagram**) yang menggambarkan struktur class, attribute, method, serta hubungan antar class dalam sistem.
3. Implementasikan relasi antar class pada program, yang mencakup:
    - *Dependency*

- *Aggregation*

2. Jelaskan perbedaan dan fungsi masing-masing jenis relasi tersebut berdasarkan kasus yang dibuat!
3. Lakukan proses generate aplikasi ke dalam file JAR!
4. Buat sebuah project Java baru yang hanya memuat file JAR hasil generate, kemudian jalankan aplikasi dari project tersebut untuk memastikan file JAR dapat digunakan dengan benar!
5. Seluruh source code, diagram kelas, serta penjelasan implementasi didokumentasikan pada jobsheet ini! Pastikan source code di submit pada repository GitHub masing-masing!

Source Code

```java
package id.ac.polban.employee.model;
public class Department {
    private static int totalDepartment = 0;

    private int id;
    private String name;

    public Department(String name) {
        totalDepartment++;
        this.id = totalDepartment;
        this.name = name;
    }

    public static int getTotalDepartment() {
        return totalDepartment;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

```java
package id.ac.polban.employee.model;

public class EmploymentType {
    private String type;

    public EmploymentType(String type) {
        this.type = type;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }
}

```

```java
package id.ac.polban.employee.model;

public class Employee {
    private static int totalEmployee = 0;

    private int id;
    private String name;
    private Department department;
    private EmploymentType type;
    private double salary;

    public Employee(String name, Department department, EmploymentType type, double salary) {
        totalEmployee++;
        this.id = totalEmployee;
        this.name = name;
        this.department = department;
        this.type = type;
        this.salary = salary;
    }

    public static int getTotalEmployee() {
        return totalEmployee;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public Department getDepartment() {
        return department;
    }

    public EmploymentType getType() {
        return type;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        if (salary >= 0) {
            this.salary = salary;
        }
    }

    public void displayInfo() {
        System.out.println("ID            : " + id);
        System.out.println("Nama          : " + name);
        System.out.println("Departement   : " + department.getName());
        System.out.println("Tipe          : " + type.getType());
        System.out.println("Gaji          : " + salary);
        System.out.println("------------------------------");
    }
}

```

```java
package id.ac.polban.employee.service;

import java.util.HashMap;
import java.util.Map;
import id.ac.polban.employee.model.Employee;

public class EmployeeService {
    private Map<Integer, Employee> employees = new HashMap<>();

    public void addEmployee(Employee emp) {
        employees.put(emp.getId(), emp);
    }

    public Employee getEmployee(int id) {
        return employees.get(id);
    }

    public void raiseSalary(int id, double percent) {
        Employee emp = employees.get(id);
        if (emp != null) {
            double newSalary = emp.getSalary() * (1 + percent / 100);
            emp.setSalary(newSalary);
        }
    }

    public void showAllEmployee() {
        for (Employee emp : employees.values()) {
            emp.displayInfo();
        }
    }
}

```

```java
package id.ac.polban.app;

import id.ac.polban.employee.model.Department;
import id.ac.polban.employee.model.Employee;
import id.ac.polban.employee.model.EmploymentType;
import id.ac.polban.employee.service.EmployeeService;

public class Main {
    public static void main(String[] args) {

        Department it = new Department("IT");
        Department hr = new Department("HR");
        Department finance = new Department("Finance");
        Department marketing = new Department("Marketing");

        EmploymentType fulltime = new EmploymentType("Full Time");
        EmploymentType contract = new EmploymentType("Contract");

        EmployeeService service = new EmployeeService();

        Employee emp1 = new Employee("Raza", it, fulltime, 5000000);
        Employee emp2 = new Employee("Aqmar", hr, contract, 4000000);
        Employee emp3 = new Employee("Bintang", finance, fulltime, 5500000);
        Employee emp4 = new Employee("Kinara", marketing, contract, 4500000);

        service.addEmployee(emp1);
        service.addEmployee(emp2);
        service.addEmployee(emp3);
        service.addEmployee(emp4);

        service.raiseSalary(1, 10);

        service.showAllEmployee();

        System.out.println("Total Employee  : " + Employee.getTotalEmployee());
        System.out.println("Total Department: " + Department.getTotalDepartment());
    }
}

```

Screenshot Hasil

Main Program

```
ID              : 1
Nama            : Raza
Departement     : IT
Tipe            : Full Time
Gaji            : 5500000.0
--------------------------------
ID              : 2
Nama            : Aqmar
Departement     : HR
Tipe            : Contract
Gaji            : 4000000.0
--------------------------------
ID              : 3
Nama            : Bintang
Departement     : Finance
Tipe            : Full Time
Gaji            : 5500000.0
--------------------------------
ID              : 4
Nama            : Kinara
Departement     : Marketing
Tipe            : Contract
Gaji            : 4500000.0
--------------------------------
Total Employee  : 4
Total Department: 4
```

Tes Jar :

```
C:\Users\BINTANG RTJSD\Documents\TUGAS TUGAS RAZA\SEMESTER 2\TEKNIK PE
MROGRAMAN\Tekpro-Week2\tesJAR>java -jar lib\Tekpro-Week2.jar
ID              : 1
Nama            : Raza
Departement     : IT
Tipe            : Full Time
Gaji            : 5500000.0
--------------------------------
ID              : 2
Nama            : Aqmar
Departement     : HR
Tipe            : Contract
Gaji            : 4000000.0
--------------------------------
ID              : 3
Nama            : Bintang
Departement     : Finance
Tipe            : Full Time
Gaji            : 5500000.0
--------------------------------
ID              : 4
Nama            : Kinara
Departement     : Marketing
Tipe            : Contract
Gaji            : 4500000.0
--------------------------------
Total Employee  : 4
Total Department: 4
```

Penjelasan Permasalahan dan Solusi

1. Penerapan Static Field dan Static Method
   Pada class Department dan Employee saya menambahkan static field:
   private static int totalDepartment;
   private static int totalEmployee;
   field ini digunakan untuk menghitung jumlah objek yang dibuat.
   Constructor akan menambah nilai static setiap kali object baru dibuat.

Saya juga membuat static method:
public static int getTotalDepartment()
public static int getTotalEmployee()
Method ini digunakan untuk mengambil total object tanpa perlu membuat object baru.
Static digunakan karena data tersebut milik class, bukan milik satu object tertentu.

2. Saya telah menerapkan konsep package dengan membuat tiga package:
   id.ac.polban.employee.model
   Berisi:
   • Department
   • Employee
   • EmploymentType

   id.ac.polban.employee.service
   Berisi:
   • EmployeeService

   Saya juga menambahkan:
   id.ac.polban.employee.app
   Berisi:
   • MainApp

   Tujuannya agar struktur program rapi dan mudah dikembangkan.
3. Diagram Kelas atau Class Diagram
   Struktur class :
   Departement :
   - id : int
   - name : String
   - totalDepartment : static int
   - getName()
   - getTotalDepartment()

   Employee :
   - id : int
   - name : String
   - department : Department
   - type : EmploymentType
   - salary : double
   - totalEmployee : static int
   - getSalary()
   - setSalary()
   - displayInfo()
   - getTotalEmployee()

   EmploymentType :
   - type : String
   - getType()

   EmployeeService :
   - employees : Map<Integer, Employee>
   - addEmployee()
   - getEmployee()
   - raiseSalary()
   - showAllEmployee()
4. Pengimplementasian relasi antar class
   Pertama adalah relasi antara Employee dengan Department dan

EmploymentType. Di dalam class Employee terdapat attribute berupa Department dan EmploymentType. Artinya setiap karyawan pasti memiliki satu departemen dan satu jenis pekerjaan. Misalnya Andi berada di departemen IT dengan tipe Full Time, sedangkan Budi berada di departemen HR dengan tipe Contract. Hubungan ini menunjukkan bahwa Employee "memiliki" Department dan EmploymentType.

Relasi tersebut termasuk aggregation. Alasannya, object Department tetap dapat berdiri sendiri walaupun object Employee dihapus. Misalnya, jika seluruh karyawan di departemen IT dihapus dari sistem, departemen IT tetap ada. Department tidak bergantung pada keberadaan Employee untuk tetap eksis. Jadi hubungan ini bersifat memiliki, tetapi tidak saling mengikat secara penuh.

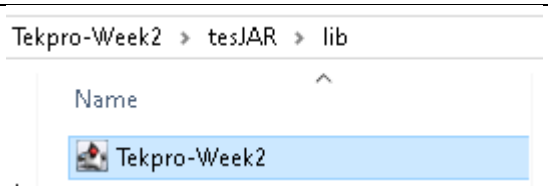
Selanjutnya terdapat relasi antara EmployeeService dan Employee. Class EmployeeService bertugas mengelola data karyawan, seperti menambahkan karyawan dan menaikkan gaji. Di dalam method seperti addEmployee dan raiseSalary, EmployeeService menggunakan object Employee sebagai parameter dan memproses datanya. Namun EmployeeService tidak menyimpan Department atau EmploymentType secara langsung sebagai bagian dari strukturnya.

Hubungan ini disebut dependency. EmployeeService bergantung pada struktur Employee untuk menjalankan fungsinya. Jika struktur atau method di class Employee berubah, misalnya nama method getSalary diubah, maka EmployeeService juga harus menyesuaikan. Jadi hubungan ini bersifat menggunakan, bukan memiliki.

5. Aggregation terlihat pada hubungan antara Employee dengan Department. Setiap Employee memiliki satu Department sebagai bagian dari datanya. Artinya, object Department disimpan sebagai attribute di dalam Employee. Namun jika semua Employee dihapus, Department tetap ada. Jadi hubungan ini bersifat memiliki, tetapi tidak saling bergantung secara penuh.

   Sedangkan dependency terlihat pada hubungan antara EmployeeService dengan Employee. EmployeeService menggunakan object Employee untuk menjalankan proses seperti menambah data atau menaikkan gaji. Namun EmployeeService tidak menyimpan Employee sebagai bagian tetap dari strukturnya. Ia hanya menggunakan Employee saat diperlukan.

   Jadi perbedaannya, aggregation menunjukkan hubungan memiliki sebagai bagian dari struktur data, sedangkan dependency hanya menunjukkan hubungan penggunaan dalam proses tertentu.
6. Generate JAR
   Untuk membuat sebuah file JAR di VSCode, hanya perlu melakukan beberapa hal. Namun pastikan sudah menginstal Java Extension Pack. Caranya adalah :
   1. Tekan Ctrl + Shift P
   2. Cari Java : Export JAR
   3. Pilih Main class nya
   4. Pilih lokasi penyimpanan
   5. File JAR akan dibuat di lokasi folder yang dipilih



7. Mencoba file JAR dengan membuat Projek baru
   Saya membuat sebuah projek java baru yang bernama tesJAR, cara mengetesnya simpel, yaitu sebagai berikut :
   - Buka VSCode
   - Buka folder projek
   - Buat folder baru bernama lib
   - Masukkan file Tekpro-Week2.JAR ke folder lib
   - Jalankan dari terminal dengan mengetik perintah java -jar lib/Tekpro-Week2.jar
   - Output ada pada Screenshot hasil

Nama Teman dan Hal yang Dibantu (Opsional)

- M Haafiz M Nayyara (Memberitahu dalam hal membuat JAR, dan sebagai teman yang bisa saya tanya tanya)
- Firon F Sahuleka (Teman yang saya tanya-tanya di beberapa hal yang saya tidak paham)